

AI Nurse Robot — Complete Build Guide

From Empty Room to Working Demo

This book is written so that anyone — including someone with no prior knowledge of this project — can pick it up and build the entire system from scratch, without needing to look anything else up. Follow the chapters in order. Do not skip ahead. Each chapter depends on the one before it.

Project summary: A JetAuto Pro robot acts as a hospital delivery assistant. A person types a command on a Mac (example: "move med 1 to bed 1"). The command travels over WiFi to the robot. The robot drives itself to a medicine shelf, uses its camera and a custom-trained AI model to identify the correct medicine, picks it up with its arm, drives to the correct bed, drops the medicine, and sends a confirmation back to the Mac.

Hardware used:

- JetAuto Pro mobile robot (with LiDAR, camera, robotic arm, running ROS)
- A Mac (or any laptop) connected to the same WiFi network as the robot

Software stack:

- ROS (Robot Operating System) — handles navigation, sensors, and the arm
 - Python — used for all custom scripts
 - A custom-trained Roboflow object detection model — identifies medicines from camera images
 - Python sockets — send commands between Mac and robot
 - Tkinter — the control window on the Mac
-

Chapter 1: Physical Setup

This chapter has zero coding. Everything here is about arranging your physical space correctly. Get this right first — every later chapter depends on the room staying exactly as you set it up here.

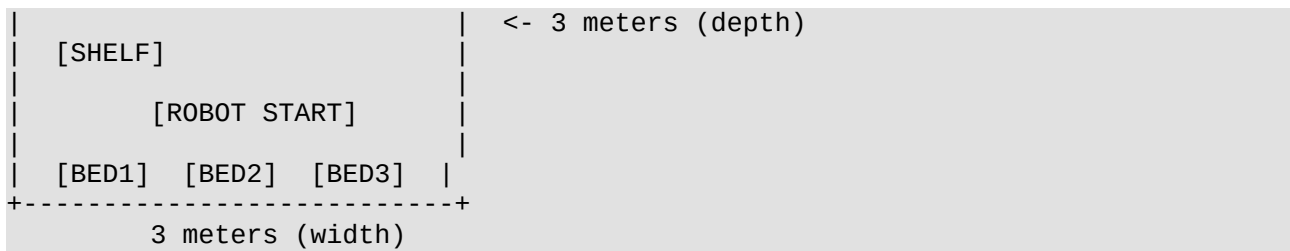
1.1 Choosing and marking the room boundary

Pick a flat, clear area of floor measuring 3 meters by 3 meters. This size is large enough for the robot to turn around freely, drive between locations, and avoid obstacles, but small enough that mapping and testing don't take too long.

Use masking tape or any tape that won't damage your floor. Mark all four corners of the 3x3 square, then run tape along all four edges so the boundary is a complete visible square on the floor. This boundary is just a visual guide for you — the robot doesn't need to "see" it, but everything you place (shelf, beds, robot start point) should stay inside it.

Here is the layout you are aiming for:

```
+-----+
```



Top of the layout = medicine shelf area. Bottom of the layout = the three beds. Middle = robot's home position.

1.2 Placing the medicine shelf

Place a shelf, table, or any flat raised surface against the top-left wall of your marked square.

"Against the wall" is important — it means the robot will always approach this shelf from the same direction (from the open side, facing the wall). This keeps the camera angle and arm position consistent every time.

Do not place the shelf in the middle of the room, and do not place it at an angle. It should sit flush against the wall, facing into the room.

Once placed, do not move it again for the rest of this project. Every later chapter — the map, the saved coordinates, the arm positions — depends on this shelf staying in exactly this spot. If you must move it later, you will need to redo Chapter 3 (mapping) and Chapter 4 (saved locations) again.

Shelf height: Choose a height that is comfortable for the robot arm to reach. JetAuto Pro arms typically reach best at a height roughly level with the robot's own body — not too high, not on the floor. If unsure, do a quick manual test: move the arm by hand (or via the control software) to a natural resting/reaching position, and set the shelf height to match where the gripper naturally sits when extended forward.

1.3 Placing the three beds

Place three beds, tables, or boxes (anything representing a "bed" for this demo) along the bottom wall of your marked square, evenly spaced. Leave at least 40 centimeters of gap between each bed so the robot has room to approach each one individually without bumping into the neighboring bed.

Label them clearly with paper signs: BED 1, BED 2, BED 3 — left to right, in that order. This is just for your own reference during testing; the robot doesn't read these signs.

Do not move these once placed, for the same reason as the shelf.

1.4 Marking the robot's starting position

Pick a spot roughly in the center of your 3x3 square. This is where the robot will sit before every single run. Mark this spot with tape in a cross or "+" shape on the floor, and also mark the direction the robot should face (for example, facing toward the shelf).

This spot matters more than it might seem. In Chapter 3, you will tell the robot's software "you are currently at this exact position" — this is called localization. If the robot doesn't start in the exact

same spot and facing the exact same direction every time, its understanding of where everything else is will be wrong, and navigation will fail or be inaccurate.

Practical tip: Take a photo of the robot sitting on this mark, facing the correct direction, from a few different angles. Keep this photo handy — it's your reference for "this is what correct startup looks like."

1.5 Preparing the three medicine items

Since your detection model (covered in Chapter 6) was trained on 3 specific medicines using 300 images from Roboflow, those exact same 3 physical medicine items (or visually identical replacements) must be used during testing and the demo. The model has learned what these specific items look like — if you swap them for different-looking boxes, detection accuracy will drop.

Place these 3 medicine items on the shelf, side by side, in a fixed left-to-right order. Decide now which is "med1", which is "med2", and which is "med3" — write this down, because this same order will be used in your configuration files later (Chapter 6 and Chapter 8).

Mark the exact spot of each item on the shelf with small tape marks (an outline of where each box sits). This matters for Chapter 7 (the arm), because the arm will use fixed, pre-recorded positions to pick up each item — if the item is shifted even a few centimeters, the arm may miss it.

1.6 Lighting

Make sure the area around the shelf has consistent, even lighting — not too dark, and no strong glare or shadows directly on the medicine items. The camera-based detection model in Chapter 6 will perform best under the same lighting conditions it was trained on (or similar). If your demo will happen at a different time of day than your testing, do a final detection test under the actual demo lighting before the real demo.

A simple desk lamp pointed at the shelf area, kept in the same position every time, is a reliable way to keep lighting consistent.

1.7 Floor surface

Make sure the floor inside your 3x3 square is flat, hard (not carpet if possible — carpet can affect wheel odometry and LiDAR readings), and completely free of loose cables, rugs, shoes, or any object that isn't part of your fixed setup (shelf, beds). Anything left on the floor becomes an "obstacle" the robot has to deal with, and if it's not on the map, it can cause navigation problems.

1.8 End-of-chapter checklist

Before moving to Chapter 2, confirm all of these:

- ☐ 3m x 3m square marked with tape on the floor
- ☐ Shelf placed against the top-left wall, facing into the room, at arm-reachable height
- ☐ Three beds placed along the bottom wall, evenly spaced, labeled BED1/BED2/BED3
- ☐ Robot starting position marked with tape, with facing direction noted

- ☐ Three medicine items placed on shelf in a fixed left-to-right order, positions marked with tape, and you have written down which is med1/med2/med3
- ☐ Lighting set up and consistent
- ☐ Floor inside the square is clear and flat

If all boxes are checked, proceed to Chapter 2.

Chapter 2: Robot Software Environment Check

Before doing anything else with software, you need to confirm the robot's existing systems all work. This chapter is mostly running commands and observing results — very little can go wrong here if the robot was working before, but it's important to verify everything in one sitting so you know your baseline is solid.

2.1 Connecting to the robot

The robot runs its own onboard computer (likely a Jetson) which you connect to over WiFi using SSH (a remote terminal connection).

On your Mac, open Terminal and connect:

```
ssh jetauto@<ROBOT_IP_ADDRESS>
```

Replace <ROBOT_IP_ADDRESS> with the robot's actual IP address on your WiFi network (you mentioned 192.168.149.1 previously — use whatever is correct for your setup). Enter the password when prompted. If this connects successfully, you're in.

[Coding Part of SSH Setup — full details] If SSH asks for a password every time and you want to skip that step in the future, you can set up SSH key-based login. This is optional and only worth doing if you're tired of typing the password repeatedly. Steps: generate a key pair on your Mac with `ssh-keygen`, then copy the public key to the robot with `ssh-copy-id jetauto@<ROBOT_IP_ADDRESS>`. After this, `ssh jetauto@<ROBOT_IP_ADDRESS>` will connect without asking for a password.

2.2 Checking ROS starts cleanly

Once connected via SSH, start the ROS master:

```
roscore
```

You should see output ending with something like `started core service [/rosout]`. Leave this running — open a second SSH session (a new Terminal tab, SSH'd in again) for all further commands, so `roscore` keeps running in the first window.

If `roscore` fails or shows errors, do not proceed — this means ROS itself isn't installed correctly, and that's outside the scope of this book (it would mean re-flashing or repairing the robot's OS image, which should have been done before you started this project).

2.3 Checking the JetAuto bringup launches correctly

In your second SSH session, run the robot's main bringup launch file. This starts all the robot's hardware drivers — wheels, LiDAR, camera, arm — together:

```
roslaunch jetauto_bringup jetauto_bringup.launch
```

Watch the output scroll by. You're looking for the absence of red `[ERROR]` lines. Some yellow `[WARN]` lines are usually normal and can be ignored. If you see errors mentioning specific hardware (e.g. "camera not found", "arm controller failed to connect"), that specific piece of hardware needs to be checked physically — cables, power, connections — before continuing.

Leave this running too. You now have two terminals open: one running `roscore`, one running `jetauto_bringup`.

2.4 Checking the arm responds

Open a third SSH session. List the available ROS topics related to the arm:

```
rostopic list | grep arm
```

You should see one or more topics related to arm joints or arm control. The exact names depend on the JetAuto software version, but common patterns include something with `joint` or `arm` in the name.

To do a simple test movement, JetAuto Pro usually ships with example scripts for moving the arm. Check what's available:

```
ls ~/jetauto_ws/src/jetauto_example/scripts/ 2>/dev/null
```

or wherever the JetAuto example scripts are installed on your specific robot (this can vary). Look for anything with "arm" in the filename — for example a demo script that moves the arm to a home position. Run it:

```
python3 <path_to_arm_demo_script>.py
```

You should physically see the arm move. If it does, your arm is working. If nothing happens, check that the arm's power and signal cables are properly connected, and that `jetauto_bringup` (from step 2.3) didn't show any arm-related errors.

2.5 Checking the camera stream

List camera-related topics:

```
rostopic list | grep camera
```

You're looking for an image topic — commonly something ending in `/image_raw` or `/image_raw/compressed`. Note down the exact topic name; you will need it in Chapter 6.

To actually view the stream from your Mac, you have two simple options:

Option A — RViz (recommended, also useful for later chapters): On your Mac, if you have ROS installed locally and it's configured to talk to the robot's ROS master (same network,

ROS_MASTER_URI environment variable pointing at the robot), open RViz:

```
rviz
```

Add an "Image" display and set its topic to the camera topic you found above. You should see a live video feed.

Option B — web-based viewer: Some JetAuto setups include a web video server. If so, you can view the camera by opening a browser on your Mac and going to something like `http://<ROBOT_IP_ADDRESS>:8080/stream?topic=/camera/rgb/image_raw` (the exact port and topic depend on your setup — check the JetAuto documentation that came with your robot for the exact address).

Either way, confirm you can see a live image from the robot's camera before moving on.

2.6 End-of-chapter checklist

- ☐ `roscore` starts with no errors and is left running
- ☐ `jetauto_bringup` launches with no red [ERROR] lines and is left running
- ☐ Arm physically moves when running a test/demo script
- ☐ Camera topic identified and live image viewable from the Mac
- ☐ You have written down: the camera image topic name (needed in Chapter 6)

If all of these pass, proceed to Chapter 3. If any fail, resolve the hardware/software issue first — every later chapter assumes these basics work.

Chapter 3: Building the Map (SLAM)

3.1 What this chapter achieves

By the end of this chapter, the robot will have a permanent 2D map of your 3x3 room saved as two files. Every future navigation command (Chapters 4, 5, 8) depends on this map existing. If the room layout changes after this chapter, this chapter must be redone.

3.2 What SLAM is, in plain terms

SLAM stands for Simultaneous Localization and Mapping. The robot has a LiDAR sensor — a spinning laser that measures distance to nearby objects in all directions, many times per second. As you drive the robot around the room, the LiDAR continuously measures the distance to the walls, the shelf, and the beds. Software on the robot stitches all these measurements together into a single 2D top-down picture of the room — this picture is the "map". At the same time, the software keeps track of where the robot itself is within that picture as it moves — this is the "localization" part.

You don't need to understand the math. What you need to know practically: drive the robot slowly and smoothly around the entire room, covering every part of it, and the map builds itself.

3.3 Starting SLAM mode

Make sure `roscore` and `jetauto_bringup` from Chapter 2 are still running in their terminals (or restart them if you've rebooted). In a new SSH session, start the SLAM mapping launch file:

```
roslaunch jetauto_slam jetauto_slam.launch
```

This starts the mapping process. It won't show much output beyond startup messages — that's normal. The mapping is happening in the background as data comes in from the LiDAR.

3.4 Watching the map build in RViz

On your Mac, open RViz (same as Chapter 2, step 2.5). Add a "Map" display and set its topic to `/map`. You should see a mostly empty/gray screen at first, with some lines appearing where the LiDAR currently sees walls near the robot's starting position.

Also add a "LaserScan" display (topic typically `/scan`) so you can see the live LiDAR readings — this helps you understand what the robot is currently "seeing" as you drive.

3.5 Driving the robot to build the map

Open one more SSH session and start keyboard teleoperation — this lets you drive the robot using your keyboard:

```
roslaunch teleop_twist_keyboard teleop_twist_keyboard.py
```

This will show you a small instruction screen with keys for moving forward, backward, and turning (commonly `i` = forward, `,` = backward, `j/l` = turn left/right, `k` = stop). Keep this terminal window focused while pressing these keys — the robot moves as you press them.

Driving strategy:

Drive slowly. Sudden fast movements can cause the LiDAR data to "smear" and create a blurry or doubled-up map.

Drive the robot along the inside edge of all four walls of your 3x3 square first — this captures the room's outer boundary clearly. Then drive past the shelf (close enough that the LiDAR clearly registers it as an object), and past each of the three beds.

Finally, drive through open areas of the room — back and forth a couple of times — so the map's open floor area is well-defined, not just the edges.

Watch RViz the whole time. As you drive, you'll see the gray map area filling in with black lines (walls/obstacles) and white space (open floor). Keep driving until:

- All four walls of your square appear as continuous black lines with no gaps
- The shelf appears as a distinct black shape
- All three beds appear as distinct black shapes
- The open floor area between them is clearly white (mapped as free space), not gray (unmapped)

This usually takes a few minutes of slow, deliberate driving. There's no harm in driving extra loops

— more data generally makes the map cleaner, as long as you're driving slowly.

3.6 Returning to start position

Once the map looks complete in RViz, drive the robot back to the exact starting position and orientation you marked in Chapter 1 (step 1.4), using the same teleop controls. This isn't strictly required for saving the map, but it's good practice — it means your next session (Chapter 4) starts from a known, consistent state.

3.7 Saving the map

With the map looking complete in RViz, save it. In a new SSH session, run:

```
roslaunch map_server map_saver -f ~/room_map
```

This creates two files in the robot's home directory:

- `room_map.pgm` — the actual map image (a grayscale picture: black = wall, white = free space, gray = unknown)
- `room_map.yaml` — a small text configuration file that tells ROS how to interpret the `.pgm` file (resolution, origin point, etc.)

3.8 Verifying the saved map

Copy the `.pgm` file to your Mac to look at it directly. From your Mac terminal:

```
scp jetauto@<ROBOT_IP_ADDRESS>:~/room_map.pgm ~/Desktop/
```

Open `room_map.pgm` on your Mac (most image viewers can open `.pgm`, or use an image editor like GIMP). You should see a clean top-down outline of your room: a roughly square black boundary, with distinct black shapes where the shelf and beds are, and white space everywhere else inside the boundary.

If something looks wrong:

- **Gaps in the walls** (white or gray where there should be a continuous black line): you didn't drive close enough to that wall, or drove too fast past it. Re-run SLAM (step 3.3) and drive more carefully near that area. You don't need to start over — SLAM continues building on the existing map as long as the process is still running. If you've already stopped it, you'll need to start again from step 3.3.
- **Doubled or blurry walls** (two parallel lines instead of one): you drove too fast at some point, causing position-tracking drift. It's often fine to leave minor blurriness — navigation in Chapter 5 can usually tolerate it. If it's severe, redo the mapping more slowly.
- **Large gray (unknown) areas in the middle of the room**: you didn't drive through that area. Re-run SLAM and drive through that section.

3.9 Locking in the map

Once you're satisfied with the map, you're done with this chapter. The two files (`room_map.pgm`

and `room_map.yaml`) are now your permanent map — keep a backup copy of both (e.g. copy them to your Mac as well, in case anything happens to the robot's storage).

From this point forward: Do not move the shelf, the beds, or change the room's physical layout. If anything changes, this chapter must be redone, and Chapter 4's saved coordinates will also need to be redone.

3.10 End-of-chapter checklist

- ☐ SLAM mapping launched successfully
- ☐ Entire room driven slowly and fully — all walls, shelf, and beds visible in RViz as distinct shapes
- ☐ Map saved as `room_map.pgm` and `room_map.yaml`
- ☐ Map visually verified — no major gaps or blank areas
- ☐ Backup copies of both map files saved to your Mac

If all boxes are checked, proceed to Chapter 4.

Chapter 4: Recording Goal Locations

4.1 What this chapter achieves

By the end of this chapter, you will have a single file called `locations.json` saved on the robot. This file contains the exact X position, Y position, and facing angle for the medicine shelf and each of the three beds, as measured on the map you created in Chapter 3. Every future navigation command reads this file to know where to go.

4.2 Starting the robot with the saved map

Make sure `roscore` is running (Chapter 2). In a new SSH session, launch the robot's navigation stack, telling it to use the map you saved in Chapter 3:

```
roslaunch jetauto_nav jetauto_nav.launch map:=$HOME/room_map.yaml
```

This starts the full navigation system: it loads your map, starts the localization system (AMCL), and starts the path-planning system (`move_base`). At this point the robot doesn't yet know exactly where it is on the map — it needs to be told.

4.3 Telling the robot where it currently is

Make sure the robot is physically sitting on the starting position you marked in Chapter 1 (step 1.4), facing the same direction as when you finished mapping.

On your Mac, open RViz. Add a "Map" display (topic `/map`) — you should now see your saved map appear. Add a "RobotModel" or at least a "TF" display so you can see where the robot thinks it is (it will likely be wrong/random at first, shown as an arrow somewhere on the map).

In RViz's toolbar, click the button labeled **"2D Pose Estimate"**. Click on the map at the exact spot

where the robot is physically sitting (this should correspond to the starting position you marked), and while holding the mouse button, drag in the direction the robot is facing, then release. This tells the localization system: "the robot is here, facing this way."

The robot's LiDAR will immediately start comparing what it currently sees to the map, and the position estimate (shown as a cloud of small arrows, which should converge into a tight cluster) will settle onto the correct spot. If the robot is in the right physical position and facing the right way, this usually settles within a second or two.

If the robot starts moving slightly on its own right after this — that's the localization system doing small corrections, which is normal. If it spins wildly or drives into a wall, your "2D Pose Estimate" click was likely in the wrong spot or wrong direction — click "2D Pose Estimate" again and try to be more precise.

4.4 Driving to the medicine shelf position and recording it

You will now manually send the robot to the position it should stop at when collecting medicine, and then read off the exact coordinates.

In RViz's toolbar, click **"2D Nav Goal"**. Click on the map at the spot directly in front of the shelf — close enough that the arm (once we get to Chapter 7) will be able to reach the shelf, but not so close that the robot's body bumps into it. Drag in the direction the robot should be facing when it arrives (it should be facing the shelf, since that's the direction the arm needs to reach). Release.

The robot will plan a path and drive there on its own — watch it go. This is your first real test of the navigation system working. Wait until it comes to a complete stop.

Now read the robot's current position. In a new SSH session:

```
rostopic echo /amcl_pose -n 1
```

This prints one message and stops (`-n 1` means "just one message"). You'll see output containing a `pose` section with `position` (`x`, `y`, `z`) and `orientation` (`x`, `y`, `z`, `w` — a quaternion representing rotation).

Write down the `position.x`, `position.y`, and the four `orientation` values. You'll convert orientation into a simple angle in the next step.

[Coding Part of Converting Orientation to a Simple Angle — full details] The `orientation` values (`x`, `y`, `z`, `w`) describe rotation in a format called a quaternion, which is hard to read directly. Since the robot only rotates around one axis (it's on a flat floor), you only care about the `z` and `w` values. Run this short Python snippet on the robot (or your Mac, if it has Python) to convert it into a simple angle in radians:

```
import math

def quaternion_to_yaw(z, w):
    return 2 * math.atan2(z, w)

# Example - replace with your actual recorded values
z = 0.0 # paste your orientation.z here
w = 1.0 # paste your orientation.w here
print("Angle in radians:", quaternion_to_yaw(z, w))
```

Run this with `python3 convert_angle.py` after filling in your actual `z` and `w` values from step 4.4's output. The printed number is the angle you'll use in `locations.json`. Repeat this conversion for each location you record in this chapter.

4.5 Driving to each bed position and recording it

Repeat the exact same process from step 4.4 for each of the three beds:

For **Bed 1**: Click "2D Nav Goal" in RViz, click on the map at the position directly in front of Bed 1 where the robot should stop to make a delivery, drag to set the facing direction (facing the bed), release, wait for arrival, then run `rostopic echo /amcl_pose -n 1` and write down the values. Convert the orientation to an angle using the snippet above.

For **Bed 2**: Repeat.

For **Bed 3**: Repeat.

By the end of this step, you should have four sets of numbers written down: one for the medicine shelf, and one for each of the three beds. Each set has an `x`, a `y`, and an angle in radians.

4.6 Creating the `locations.json` file

On the robot, create a file called `locations.json` in the robot's home directory with your recorded values. You can create this file directly using a text editor available over SSH (e.g. `nano`):

```
nano ~/locations.json
```

Type or paste the following structure, replacing the example numbers with your actual recorded values from steps 4.4 and 4.5:

```
{
  "medicine_shelf": {"x": 1.20, "y": 2.50, "angle": 0.0},
  "bed_1": {"x": 2.80, "y": 0.40, "angle": 1.57},
  "bed_2": {"x": 2.80, "y": 1.50, "angle": 1.57},
  "bed_3": {"x": 2.80, "y": 2.60, "angle": 1.57}
}
```

Save and exit (`nano` shortcut: `Ctrl+O` then `Enter` to save, `Ctrl+X` to exit).

4.7 Understanding the angle values

The angle is in radians, and represents which direction the robot faces when it arrives:

- `0.0` — facing along the positive `X` direction of the map (often "right" on the RViz screen)
- `1.57` (approximately $\pi/2$) — facing along the positive `Y` direction ("up" on the RViz screen)
- `3.14` (approximately π) — facing the opposite of `0.0` ("left")
- `-1.57` — facing the opposite of `1.57` ("down")

These are approximate, and the exact direction depends on how your map is oriented. What matters in practice is: when the robot arrives at the shelf, it should be facing the shelf (so the arm in Chapter

7 can reach it). When it arrives at a bed, it should be facing the bed (so the arm can place the medicine there). If your recorded angle from `/amcl_pose` doesn't match what you expected, that's fine — use the actual recorded value, since that reflects the real direction you physically drove the robot to face.

4.8 Verifying the file

Double check the file is valid by displaying it:

```
cat ~/locations.json
```

Confirm all four locations are present, each with `x`, `y`, and `angle` as numbers (not text, no quotes around the numbers).

4.9 End-of-chapter checklist

- ☐ Robot successfully localized on the saved map using "2D Pose Estimate"
- ☐ Robot successfully navigated to the medicine shelf position via "2D Nav Goal"
- ☐ Robot successfully navigated to Bed 1, Bed 2, and Bed 3 positions
- ☐ All four positions' `x`, `y` values recorded from `/amcl_pose`
- ☐ All four orientation values converted to angles in radians
- ☐ `locations.json` created on the robot with all four locations, verified with `cat`

If all boxes are checked, proceed to Chapter 5.

Chapter 5: Autonomous Navigation Function

5.1 What this chapter achieves

By the end of this chapter, you will have a single reusable function, `go_to(location_name)`, that drives the robot to any of the four saved locations completely automatically — no manual clicking in RViz needed. This function is the building block that Chapter 8 (full pipeline) will call repeatedly.

5.2 How this works conceptually

The `move_base` system you've already been using indirectly (every time you clicked "2D Nav Goal" in Chapter 4) can also be controlled directly from a Python script using something called `actionlib`. Instead of you clicking a point on a map, your script sends the `x`, `y`, and `angle` values directly to `move_base`, and then waits for `move_base` to report back "goal reached" or "goal failed".

Your `go_to()` function will: look up the coordinates for the requested location name in `locations.json`, send those coordinates to `move_base`, and wait for the result.

5.3 Writing the navigator script

[Coding Part of Autonomous Navigation Function — full details]

On the robot, create a file called `navigator.py`:

```
nano ~/navigator.py
```

Paste the following code:

```
#!/usr/bin/env python3
import rospy
import actionlib
import json
import math
import sys
from move_base_msgs.msg import MoveBaseAction, MoveBaseGoal
from geometry_msgs.msg import Quaternion

LOCATIONS_FILE = "/home/jetauto/locations.json"

def load_locations():
    with open(LOCATIONS_FILE, "r") as f:
        return json.load(f)

def angle_to_quaternion(angle):
    # Converts a simple yaw angle (radians) into the quaternion format ROS
    # expects
    return Quaternion(
        x=0.0,
        y=0.0,
        z=math.sin(angle / 2.0),
        w=math.cos(angle / 2.0)
    )

def go_to(location_name):
    locations = load_locations()
    if location_name not in locations:
        print("ERROR: Unknown location '%s'" % location_name)
        return False

    loc = locations[location_name]

    client = actionlib.SimpleActionClient('move_base', MoveBaseAction)
    print("Waiting for move_base server...")
    client.wait_for_server()

    goal = MoveBaseGoal()
    goal.target_pose.header.frame_id = "map"
    goal.target_pose.header.stamp = rospy.Time.now()
    goal.target_pose.pose.position.x = loc["x"]
    goal.target_pose.pose.position.y = loc["y"]
    goal.target_pose.pose.position.z = 0.0
    goal.target_pose.pose.orientation = angle_to_quaternion(loc["angle"])

    print("Sending robot to '%s' (x=%.2f, y=%.2f, angle=%.2f)" % (
        location_name, loc["x"], loc["y"], loc["angle"]))

    client.send_goal(goal)
    finished = client.wait_for_result(timeout=rospy.Duration(60))

    if not finished:
        print("ERROR: Timed out reaching '%s'" % location_name)
```

```

        return False

    state = client.get_state()
    if state == actionlib.GoalStatus.SUCCEEDED:
        print("OK: Arrived at '%s'" % location_name)
        return True
    else:
        print("ERROR: Failed to reach '%s' (state=%s)" % (location_name, state))
        return False

if __name__ == "__main__":
    rospy.init_node("navigator_node", anonymous=True)
    if len(sys.argv) < 2:
        print("Usage: python3 navigator.py <location_name>")
        sys.exit(1)
    go_to(sys.argv[1])

```

Save and exit. Make the file executable:

```
chmod +x ~/navigator.py
```

Important: Check the `LOCATIONS_FILE` path at the top of the script matches where your `locations.json` actually is. If your username on the robot isn't `jetauto`, change `/home/jetauto/locations.json` to the correct path (you can check with `pwd` and `whoami` after SSHing in).

5.4 Testing each location individually

Make sure `roscore`, `jetauto_bringup`, and `jetauto_nav` (from Chapter 4, step 4.2) are all running, and the robot is localized (Chapter 4, step 4.3).

Test the medicine shelf:

```
python3 ~/navigator.py medicine_shelf
```

Watch the robot drive there on its own. You should see `OK: Arrived at 'medicine_shelf'` printed when it's done.

Now test each bed:

```
python3 ~/navigator.py bed_1
python3 ~/navigator.py bed_2
python3 ~/navigator.py bed_3
```

5.5 Reliability testing

Run each of the four tests three times in a row (you can run them in any order, but test each location at least 3 times total). For each run, check:

- Did the robot print `OK: Arrived at...`?
- Did it physically stop in the correct position, facing the correct direction (close enough that the arm would be able to reach the shelf/bed in the next chapters)?

If a test fails or the robot stops in the wrong place:

- If it says `ERROR: Failed to reach...` — the path may be blocked by something not

on the map (check the floor is clear), or the goal position might be too close to a wall/obstacle for `move_base` to consider it reachable. Try re-recording that location's coordinates slightly further from the obstacle (redo Chapter 4 step 4.4/4.5 for just that location).

- If it says OK but the robot is in a noticeably different spot than expected — the localization may have drifted. Make sure you started from the correct robot starting position (Chapter 1, step 1.4) and re-did "2D Pose Estimate" (Chapter 4, step 4.3) for this session.
- If the robot consistently lands a few centimeters off in the same direction every time — this is fine and expected; `move_base` has a default tolerance (usually a few centimeters and a few degrees) for what counts as "arrived". If this small offset is too large for the arm to work correctly in Chapter 7, you can re-record that location's coordinates with a small manual offset to compensate.

5.6 End-of-chapter checklist

- ☐ `navigator.py` created on the robot with the correct `LOCATIONS_FILE` path
- ☐ `go_to("medicine_shelf")` works via `python3 navigator.py medicine_shelf`
- ☐ `go_to("bed_1")`, `go_to("bed_2")`, `go_to("bed_3")` all work
- ☐ Each location tested at least 3 times, arriving correctly each time (or coordinates adjusted until reliable)

If all boxes are checked, proceed to Chapter 6.

Chapter 6: Medicine Detection with Your Custom Roboflow Model

6.1 What this chapter achieves

By the end of this chapter, you will have a script that captures an image from the robot's camera and runs it through your custom-trained Roboflow model (300 images, 3 medicines, 90%+ accuracy) to determine which medicine(s) are visible. This is the main AI/ML component of your capstone.

6.2 Your model

You trained this model at <https://universe.roboflow.com/agenttrain/agent> using 300 custom images covering 3 medicines. It already works well — this chapter is about getting it running on the robot's camera feed, not about training (training is already done).

You will need two pieces of information from your Roboflow account, which you should locate now and write down:

- **API key** — found in your Roboflow account settings

- **Model ID and version number** — based on your project, this is in the form `workspace/project` and a version number, e.g. `agenttrain/agent` and version `1` (or whichever version number is your trained, working model)

6.3 Two ways to run the model — choosing one

Option A — Roboflow Hosted API: Your robot sends each captured image over the internet to Roboflow's servers, which run the model and send back the result. This requires the robot to have internet access at the time of detection.

Option B — Local inference: The model runs directly on the robot's own onboard computer, using a downloaded copy of your trained model. No internet needed at detection time.

Recommendation: use Option B for your actual demo. Live demos that depend on internet connectivity (Option A) risk failing due to WiFi issues at the worst possible moment. However, Option A is simpler to set up first, so it's worth testing with Option A first to confirm your model and camera integration work, then switching to Option B before the final demo.

6.4 Setting up Option A (Hosted API) — for initial testing

On the robot, install the Roboflow Python package:

```
pip3 install roboflow
```

[Coding Part of Roboflow Hosted API Setup — full details]

Create a test script `~/detect_test.py`:

```
#!/usr/bin/env python3
from roboflow import Roboflow

API_KEY = "YOUR_API_KEY_HERE"
WORKSPACE = "agenttrain"
PROJECT = "agent"
VERSION = 1 # use your actual trained version number

rf = Roboflow(api_key=API_KEY)
project = rf.workspace(WORKSPACE).project(PROJECT)
model = project.version(VERSION).model

# Test on a saved image file first
result = model.predict("test_image.jpg", confidence=40, overlap=30).json()
print(result)
```

Take a test photo with the robot camera, save it as `test_image.jpg` on the robot (you can do this from a frame capture, covered in step 6.6, or temporarily by taking a photo another way and copying it over with SCP), then run:

```
python3 ~/detect_test.py
```

You should see a JSON output containing a `predictions` list, with each prediction showing a `class` (the medicine name, e.g. `med1`), a `confidence` value (e.g. `0.93`), and bounding box coordinates. If this works, your model and API key are correctly configured.

6.5 Setting up Option B (Local Inference) — for the demo

Install the local inference package:

```
pip3 install inference
```

[Coding Part of Local Inference Setup — full details]

Modify the model-loading part of your script to use local inference instead:

```
from inference import get_model

API_KEY = "YOUR_API_KEY_HERE"
model = get_model(model_id="agent/1", api_key=API_KEY) # use your
workspace/project and version

result = model.infer("test_image.jpg")[0]
print(result)
```

The first time you run this, the package will download your model's weights to the robot (this requires internet once, during setup — but not during the actual demo afterward). After that initial download, this runs fully offline.

Run this test script the same way as Option A and confirm you get prediction results back. Note that the result format from `inference` may look slightly different from the Roboflow hosted API's format — check the printed output and note the field names for `class/class_name` and `confidence`, as you'll use these in step 6.7.

6.6 Capturing a live frame from the robot's camera

Instead of a saved test image, your real detection script needs to grab a live frame from the camera topic you identified in Chapter 2 (step 2.5).

[Coding Part of Camera Frame Capture — full details]

Install the ROS-to-OpenCV bridge if not already present:

```
pip3 install opencv-python
sudo apt install ros-noetic-cv-bridge # adjust 'noetic' to your ROS version if
different
```

Here is the capture function:

```
import rospy
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

bridge = CvBridge()
CAMERA_TOPIC = "/camera/rgb/image_raw" # use the exact topic name from Chapter
2

def capture_frame(save_path="/tmp/frame.jpg"):
    msg = rospy.wait_for_message(CAMERA_TOPIC, Image, timeout=5)
    frame = bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
    cv2.imwrite(save_path, frame)
    return save_path
```

This function waits for one camera frame, converts it to a normal image format, saves it as a `.jpg` file, and returns the file path — ready to be fed into your model.

6.7 Combining capture and detection into one function

[Coding Part of Full Detection Function — full details]

Create the final detection script `~/detector.py`:

```
#!/usr/bin/env python3
import rospy
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
from inference import get_model

API_KEY = "YOUR_API_KEY_HERE"
MODEL_ID = "agent/1" # your workspace/project/version
CAMERA_TOPIC = "/camera/rgb/image_raw" # from Chapter 2
CONFIDENCE_THRESHOLD = 0.7 # adjust between 0.7 and 0.9 based on testing

bridge = CvBridge()
model = get_model(model_id=MODEL_ID, api_key=API_KEY)

def capture_frame(save_path="/tmp/frame.jpg"):
    msg = rospy.wait_for_message(CAMERA_TOPIC, Image, timeout=5)
    frame = bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
    cv2.imwrite(save_path, frame)
    return save_path

def detect_medicine(target_name):
    frame_path = capture_frame()
    result = model.infer(frame_path)[0]

    for prediction in result.predictions:
        detected_name = prediction.class_name.lower()
        confidence = prediction.confidence
        print("Detected: %s (%.1f%%)" % (detected_name, confidence * 100))

        if detected_name == target_name.lower() and confidence >=
CONFIDENCE_THRESHOLD:
            return True

    return False

def scan_for_medicine(target_name, max_attempts=5):
    for attempt in range(1, max_attempts + 1):
        print("Attempt %d: looking for %s..." % (attempt, target_name))
        if detect_medicine(target_name):
            print("Found %s on attempt %d" % (target_name, attempt))
            return True
        rospy.sleep(1)
    print("Did not find %s after %d attempts" % (target_name, max_attempts))
    return False

if __name__ == "__main__":
    import sys
    rospy.init_node("detector_node", anonymous=True)
    if len(sys.argv) < 2:
        print("Usage: python3 detector.py <target_medicine_name>")
        sys.exit(1)
    found = scan_for_medicine(sys.argv[1])
```

```
print("RESULT:", "FOUND" if found else "NOT FOUND")
```

Note on field names: depending on the exact version of the `inference` package, `prediction.class_name` and `prediction.confidence` might instead be accessed as `prediction["class"]` and `prediction["confidence"]` (dictionary-style instead of object-style). Check the printed output from your step 6.5 test to see which format your version returns, and adjust this script accordingly if needed.

6.8 Testing detection for each medicine

With the robot positioned in front of the shelf (drive it there using `navigator.py` from Chapter 5, or manually for now), test each medicine individually. Place only `med1` on the shelf (or point the camera at it) and run:

```
python3 ~/detector.py med1
```

You should see detection attempts printed, and `RESULT: FOUND`. Repeat for `med2` and `med3` individually.

Then test with all three medicines visible at once — since your model handles multi-object detection, run each of the three target names again with all three items present, and confirm each is still found correctly:

```
python3 ~/detector.py med1
python3 ~/detector.py med2
python3 ~/detector.py med3
```

6.9 Tuning the confidence threshold

If detections are inconsistent (sometimes found, sometimes not, for the same physical setup), try lowering `CONFIDENCE_THRESHOLD` in `detector.py` from `0.7` toward `0.5`, and re-test. If you get false positives (detecting the wrong medicine confidently), raise the threshold instead, toward `0.85` or `0.9`. Since your model already performs at 90%+ accuracy in general testing, `0.7` is a reasonable starting point that should rarely need much adjustment.

6.10 End-of-chapter checklist

- ☐ Roboflow API key, workspace, project, and version number written down
- ☐ Option A (hosted API) tested successfully on a sample image
- ☐ Option B (local inference) set up and tested successfully on a sample image
- ☐ Camera frame capture function working — produces a usable image file
- ☐ `detector.py` created, combining capture + local inference
- ☐ `med1`, `med2`, `med3` each individually detected correctly
- ☐ All three detected correctly when all three items are present together
- ☐ Confidence threshold tuned and stable across repeated tests

If all boxes are checked, proceed to Chapter 7.

Chapter 7: Arm Pick and Drop

7.1 What this chapter achieves

By the end of this chapter, you will have two functions, `pick(medicine_name)` and `drop.bed_name)`, that move the robot's arm to pick up a specific medicine from its fixed shelf position and release it at a bed. These functions use pre-recorded, fixed arm positions — the arm always moves the exact same way every time, which is reliable as long as the physical setup from Chapter 1 hasn't changed.

7.2 Why fixed positions instead of vision-guided picking

A more advanced robot might use its camera to visually locate exactly where to grip an object in real time. That approach is significantly more complex and is not necessary for this project, because in Chapter 1 you fixed the exact position of each medicine item on the shelf with tape marks, and in Chapter 4/5 the robot reliably arrives at the same shelf position every time. Given both of those are fixed, the arm's movement to reach each item can also be fixed and pre-recorded once.

The camera (Chapter 6) is still essential — it tells you *which* medicine is present, so you know *which* pre-recorded arm position to use. But the arm movement itself doesn't need to "see" anything; it just replays a recording.

7.3 How JetAuto arm control works

The JetAuto Pro arm has multiple joints (commonly 4-6, depending on the model), each of which can be commanded to a specific angle. A "position" for the arm is simply a list of angle values, one per joint, plus a gripper open/closed state. Moving the arm to a position means commanding all joints to those angle values, usually over a short duration (e.g. 1 second) so the movement is smooth rather than instant.

JetAuto Pro typically provides a controller interface (either a Python class or a set of ROS topics/services) for sending these joint commands. The exact interface name can vary by software version — check your robot's example scripts directory from Chapter 2, step 2.4, for an arm control example script. Use that script's method of commanding the arm as your template for the functions below.

7.4 Recording the required positions

You need to record the following positions, in order, for **each of the 3 medicines** (since each medicine sits in a different spot on the shelf — see Chapter 1, step 1.5), plus shared positions for the beds.

Per-medicine positions (x3 — one set for med1, one for med2, one for med3):

1. **Approach position** — arm extended toward the shelf, positioned just above and in front of where this specific medicine item sits, gripper open

2. **Grip position** — arm lowered slightly so the gripper surrounds the item, ready to close
3. **Lift position** — same as grip position but gripper now closed around the item, arm lifted slightly off the shelf

Shared positions (used for all medicines):

1. **Carry position** — arm folded into a safe, compact position close to the robot's body, holding the item securely, used while the robot drives between locations
2. **Drop-approach position** — arm extended toward the bed area, item still gripped, positioned just above where the item should be placed
3. **Release position** — same as drop-approach but gripper open, item released

Per-bed adjustment: Since the 3 beds may be at slightly different distances/angles from where the robot stops (even though the robot tries to arrive at the same relative position each time), you may find that a single "drop-approach" and "release" position works for all 3 beds, since the robot itself moves to face each bed and the arm just extends forward the same way regardless of which bed it's facing. Test this assumption in step 7.7 — if the drop position works for Bed 1 but not Bed 2/3, you may need a separate drop position recorded per bed instead.

7.5 Recording process

To record a position, you move the arm by hand or via a manual control interface to the desired position, then read out and write down the current joint angle values.

[Coding Part of Reading Current Arm Joint Angles — full details]

Most JetAuto setups expose the current joint angles on a ROS topic. Find it:

```
rostopic list | grep joint
```

Echo the current values while you manually position the arm (either by hand, if the arm can be moved freely when motors are unpowered/in a "compliant" mode, or using a manual jog interface if your JetAuto software provides one — check the example scripts directory again for a "manual control" or "teach mode" example):

```
rostopic echo /joint_states -n 1
```

This prints the current angle of each joint. Move the arm to each of the positions described in step 7.4, one at a time, and after each one, run the echo command and write down the full set of joint angles. Label each recording clearly, e.g. "med1_approach: [list of angles]", "med1_grip: [list of angles]", and so on for all positions across all 3 medicines plus the shared carry/drop/release positions.

This is the most time-consuming manual step in the whole project. Take your time — accuracy here directly determines how reliable Chapter 9's testing will be.

7.6 Saving recorded positions to a file

On the robot, create `~/arm_positions.json`:

```
nano ~/arm_positions.json
```

Structure it like this, filling in your actual recorded joint angle arrays (the example arrays below show 5 joint values — adjust the number of values to match however many joints your arm actually has):

```
{
  "med1_approach": [0.0, -0.5, 0.8, 0.0, 0.0],
  "med1_grip":     [0.0, -0.3, 0.6, 0.0, 0.0],
  "med1_lift":     [0.0, -0.6, 0.9, 0.0, 0.0],

  "med2_approach": [0.2, -0.5, 0.8, 0.0, 0.0],
  "med2_grip":     [0.2, -0.3, 0.6, 0.0, 0.0],
  "med2_lift":     [0.2, -0.6, 0.9, 0.0, 0.0],

  "med3_approach": [-0.2, -0.5, 0.8, 0.0, 0.0],
  "med3_grip":     [-0.2, -0.3, 0.6, 0.0, 0.0],
  "med3_lift":     [-0.2, -0.6, 0.9, 0.0, 0.0],

  "carry":         [0.0, 0.0, 0.0, 0.0, 0.0],
  "drop_approach": [0.0, -0.4, 0.7, 0.0, 0.0],
  "release":       [0.0, -0.2, 0.5, 0.0, 0.0]
}
```

Also include separate gripper open/close values if your robot's gripper is controlled independently from the arm joints — add two more entries such as "gripper_open": <value> and "gripper_closed": <value> if needed, based on what you observed in step 7.5.

7.7 Writing the pick and drop functions

[Coding Part of Arm Pick and Drop Functions — full details]

Create `~/arm_control.py`. The exact method of sending joint commands depends on your JetAuto software's arm interface (Chapter 2, step 2.4's example script shows the correct method for your robot) — the structure below uses a placeholder function

`send_arm_command(joint_angles, duration)` which you should replace with the actual method from your example script.

```
#!/usr/bin/env python3
import json
import time

POSITIONS_FILE = "/home/jetauto/arm_positions.json"

def load_positions():
    with open(POSITIONS_FILE, "r") as f:
        return json.load(f)

POSITIONS = load_positions()

def send_arm_command(joint_angles, duration=1.0):
    # Replace this with your robot's actual arm command method,
    # based on the example script found in Chapter 2, step 2.4
    raise NotImplementedError("Fill in with your robot's arm control method")

def set_gripper(open_gripper=True):
    # Replace with your robot's gripper control method
    raise NotImplementedError("Fill in with your robot's gripper control method")

def pick(medicine_name):
```

```

    approach = POSITIONS[medicine_name + "_approach"]
    grip = POSITIONS[medicine_name + "_grip"]
    lift = POSITIONS[medicine_name + "_lift"]
    carry = POSITIONS["carry"]

    set_gripper(open_gripper=True)
    send_arm_command(approach, duration=1.5)
    time.sleep(1.5)

    send_arm_command(grip, duration=1.0)
    time.sleep(1.0)

    set_gripper(open_gripper=False)
    time.sleep(0.5)

    send_arm_command(lift, duration=1.0)
    time.sleep(1.0)

    send_arm_command(carry, duration=1.5)
    time.sleep(1.5)

    print("OK: picked %s" % medicine_name)

def drop():
    drop_approach = POSITIONS["drop_approach"]
    release = POSITIONS["release"]
    carry = POSITIONS["carry"]

    send_arm_command(drop_approach, duration=1.5)
    time.sleep(1.5)

    send_arm_command(release, duration=1.0)
    time.sleep(1.0)

    set_gripper(open_gripper=True)
    time.sleep(0.5)

    send_arm_command(carry, duration=1.5)
    time.sleep(1.5)

    print("OK: dropped item")

```

Note that `drop()` doesn't take a bed name as a parameter in this version, based on the assumption from step 7.4 that one drop position works for all beds (since the robot itself repositions for each bed). If testing in step 7.9 shows this assumption is wrong, you can extend `drop()` to accept a bed name and look up a per-bed "bed1_drop_approach" etc. in `arm_positions.json` instead.

7.8 Filling in `send_arm_command` and `set_gripper`

Open your Chapter 2, step 2.4 example script and find the lines that send a joint command — it will look something like calling a method on a controller object, or publishing a ROS message to a joint command topic. Copy that exact method into `send_arm_command`, replacing the placeholder, so that calling `send_arm_command([0.0, -0.5, 0.8, 0.0, 0.0], duration=1.0)` actually moves the real arm to those joint angles over 1 second.

Do the same for `set_gripper` — find how the example script opens/closes the gripper (often this is just one more joint value, or a separate command), and wire that into `set_gripper`.

7.9 Testing pick and drop

With the robot positioned at the medicine shelf (use `navigator.py` from Chapter 5 to get it there reliably) and `med1` placed at its marked spot (Chapter 1, step 1.5), test:

```
python3 -c "from arm_control import pick, drop; pick('med1')"
```

Watch the arm carefully. It should: open the gripper, move toward the shelf, lower onto the item, close the gripper, lift, and fold into the carry position — now holding `med1`.

While the arm holds the item, drive the robot to a bed using `navigator.py`, then test:

```
python3 -c "from arm_control import pick, drop; drop()"
```

The arm should extend toward the bed, open the gripper to release the item, and fold back to carry position.

Repeat this full pick-and-drop cycle 5 times for `med1`. It must succeed all 5 times — item is gripped securely on `pick()` and lands in the bed area (not dropped early, not missed) on `drop()` — before moving on. Then repeat the same 5-times test for `med2` and `med3`.

If pick fails (misses the item, or grips air): the approach/grip joint angles for that medicine are slightly off relative to the item's actual position. Re-do step 7.5 for that medicine's positions, making small adjustments, and update `arm_positions.json`.

If the item slips out of the gripper during carry or drive: the gripper-closed value in `set_gripper/arm_positions.json` may not be tight enough. Adjust to a tighter closed value and re-test — but be careful not to set it so tight that it could damage the gripper motor or the item.

If drop releases the item outside the bed area: adjust the `drop_approach/release` joint angles, or — if this varies noticeably bed-to-bed as discussed in step 7.4 — record separate drop positions per bed.

7.10 End-of-chapter checklist

- ☐ All required positions recorded for `med1`, `med2`, `med3`, and shared carry/drop/release, saved in `arm_positions.json`
- ☐ `send_arm_command` and `set_gripper` filled in with your robot's actual control method
- ☐ `pick('med1')` succeeds 5/5 times
- ☐ `pick('med2')` succeeds 5/5 times
- ☐ `pick('med3')` succeeds 5/5 times
- ☐ `drop()` succeeds 5/5 times for each medicine, landing correctly in the bed area
- ☐ Confirmed whether one shared drop position works for all 3 beds, or per-bed positions are needed (and recorded if so)

If all boxes are checked, proceed to Chapter 8.

Chapter 8: Command Parser and Full Delivery Sequence

8.1 What this chapter achieves

This chapter ties together everything from Chapters 5, 6, and 7 into one function: given a text command like "move med1 to bed1", the robot automatically navigates to the shelf, detects and picks the correct medicine, navigates to the correct bed, and drops it — then reports success or failure as a text message. This is the function that Chapter 10 (Mac connection) will call.

8.2 Command format

To keep parsing simple and reliable, commands sent from the Mac will always follow this exact format:

```
move medX to bedY
```

Where X is 1, 2, or 3 (the medicine number) and Y is 1, 2, or 3 (the bed number). Examples: move med1 to bed1, move med2 to bed3, move med3 to bed2.

This rigid format means parsing is just splitting the text by spaces — no AI or complex language processing needed, and no ambiguity about what was meant.

8.3 Writing the parser

[Coding Part of Command Parser — full details]

Create ~/parser.py:

```
#!/usr/bin/env python3

def parse_command(text):
    text = text.strip().lower()
    parts = text.split()

    # Expected format: ["move", "medX", "to", "bedY"]
    if len(parts) != 4:
        return None, "ERROR: Invalid format. Use: move medX to bedY"

    if parts[0] != "move" or parts[2] != "to":
        return None, "ERROR: Invalid format. Use: move medX to bedY"

    medicine = parts[1] # e.g. "med1"
    destination = parts[3] # e.g. "bed1"

    valid_medicines = ["med1", "med2", "med3"]
    valid_beds = ["bed1", "bed2", "bed3"]

    if medicine not in valid_medicines:
        return None, "ERROR: Unknown medicine '%s'. Must be med1, med2, or med3"
    % medicine

    if destination not in valid_beds:
```

```

        return None, "ERROR: Unknown destination '%s'. Must be bed1, bed2, or
bed3" % destination

    return {"medicine": medicine, "destination": destination}, None

```

This returns either a dictionary with `medicine` and `destination`, or `None` plus an error message explaining what was wrong.

8.4 Note on location naming consistency

You'll notice `locations.json` (Chapter 4) uses keys like `bed_1` (with an underscore), while the parser above produces `bed1` (no underscore) from the command text. Pick one convention and make sure it's used consistently everywhere — either:

- Update `locations.json` to use `bed1`, `bed2`, `bed3` (no underscore) to match the parser, **or**
- Update the parser to output `bed_1`, `bed_2`, `bed_3` (with underscore) to match `locations.json`

Whichever you choose, also make sure `arm_positions.json` (Chapter 7) medicine names (`med1`, `med2`, `med3`) match the parser's output exactly — these already match as written above. Double-check all three files (`locations.json`, `arm_positions.json`, `parser.py`) use exactly the same naming before continuing.

8.5 Writing the full delivery sequence

[Coding Part of Full Delivery Sequence — full details]

Create `~/delivery.py`, which imports and combines everything from previous chapters:

```

#!/usr/bin/env python3
import rospy
from navigator import go_to
from detector import scan_for_medicine
from arm_control import pick, drop
from parser import parse_command

def execute_delivery(command_text):
    parsed, error = parse_command(command_text)
    if error:
        return error

    medicine = parsed["medicine"] # e.g. "med1"
    destination = parsed["destination"] # e.g. "bed1"

    # Step 1: go to medicine shelf
    print("Step 1: navigating to medicine shelf")
    if not go_to("medicine_shelf"):
        return "ERROR: Could not reach medicine shelf"

    # Step 2: scan for the requested medicine
    print("Step 2: scanning for %s" % medicine)
    if not scan_for_medicine(medicine):
        return "ERROR: Could not detect %s on shelf" % medicine

    # Step 3: pick it up

```

```

print("Step 3: picking %s" % medicine)
pick(medicine)

# Step 4: go to destination bed
print("Step 4: navigating to %s" % destination)
if not go_to(destination):
    return "ERROR: Could not reach %s (item still held)" % destination

# Step 5: drop it
print("Step 5: dropping at %s" % destination)
drop()

return "OK: Delivered %s to %s" % (medicine, destination)

if __name__ == "__main__":
    import sys
    rospy.init_node("delivery_node", anonymous=True)
    if len(sys.argv) < 2:
        print("Usage: python3 delivery.py \"move med1 to bed1\")
        sys.exit(1)
    command_text = " ".join(sys.argv[1:])
    result = execute_delivery(command_text)
    print(result)

```

This script prints each step as it happens — useful for watching the robot's progress and for debugging if something goes wrong.

8.6 Testing the full sequence end-to-end

With everything from Chapters 1-7 set up, and the robot at its starting position (Chapter 1, step 1.4) with `med1` placed at its marked shelf spot, run:

```
python3 ~/delivery.py "move med1 to bed1"
```

Watch the entire sequence happen automatically: drive to shelf → detect med1 → pick med1 → drive to bed1 → drop. The final printed line should be:

```
OK: Delivered med1 to bed1
```

8.7 Testing error handling

Test what happens with an invalid command:

```
python3 ~/delivery.py "move med5 to bed1"
```

This should immediately print an error from the parser (`ERROR: Unknown medicine 'med5' . . .`) without the robot moving at all — confirming the parser correctly rejects bad input before any physical action happens.

Also test what happens if the requested medicine isn't actually on the shelf (remove `med2` from the shelf, then run a command requesting `med2`):

```
python3 ~/delivery.py "move med2 to bed1"
```

The robot should still navigate to the shelf, attempt detection (Chapter 6's retry logic will try up to 5 times), then return `ERROR: Could not detect med2 on shelf` — and importantly, it

should not attempt to pick anything or proceed to a bed.

8.8 End-of-chapter checklist

- ☐ `parser.py` created and correctly validates/rejects command formats
- ☐ Naming convention (`bed1` vs `bed_1`, etc.) made consistent across `locations.json`, `arm_positions.json`, and `parser.py`
- ☐ `delivery.py` created, importing `navigator`, `detector`, `arm_control`, and `parser`
- ☐ `python3 delivery.py "move med1 to bed1"` runs the full sequence successfully end-to-end
- ☐ Invalid command correctly rejected by parser before any robot movement
- ☐ Missing-medicine scenario correctly returns an error after retries, without proceeding to pick/drop

If all boxes are checked, proceed to Chapter 9.

Chapter 9: Connecting the Mac to the Robot

9.1 What this chapter achieves

By the end of this chapter, you can type a command into a window on your Mac, and the robot will receive it, run the full delivery sequence from Chapter 8, and send back a confirmation message that appears on your Mac. This is the final piece connecting everything together.

9.2 Overall connection design

Your Mac runs `chat_client.py` (the socket client). The robot runs `chat_server.py` (the socket server). They connect once over WiFi and stay connected. The Mac sends a command string; the robot runs `execute_delivery()` from Chapter 8 and sends back the result string; the Mac displays it.

Because the robot is busy for the entire duration of a delivery (potentially 1-2 minutes), the Mac side must wait for a response before allowing another command to be sent — this is called a "busy lock", covered in step 9.5.

9.3 Updating the robot's server script

[Coding Part of Robot Server Script — full details]

On the robot, create or update `~/chat_server.py`:

```
#!/usr/bin/env python3
import socket
import rospy
from delivery import execute_delivery

def handle_client(conn, addr):
    print("Connected: %s" % str(addr))
```

```

try:
    while True:
        data = conn.recv(1024)
        if not data:
            print("Mac disconnected")
            break

        command_text = data.decode("utf-8").strip()
        print("Received command: %s" % command_text)

        result = execute_delivery(command_text)
        print("Result: %s" % result)

        conn.send(result.encode("utf-8"))
except Exception as e:
    print("Error: %s" % str(e))
finally:
    conn.close()

def start_server():
    host = "0.0.0.0"
    port = 5050

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((host, port))
    server.listen(1)
    print("Waiting for connection on port %d..." % port)

    while True:
        conn, addr = server.accept()
        handle_client(conn, addr)
        # after a client disconnects, loop back and wait for a new connection

if __name__ == "__main__":
    rospy.init_node("chat_server_node", anonymous=True)
    start_server()

```

This is similar in structure to your earlier `chat_server.py`, but now each received command is passed to `execute_delivery()` from Chapter 8 instead of just echoing or printing.

9.4 Updating the Mac's client script

[Coding Part of Mac Client Script — full details]

On your Mac, create or update `chat_client.py`:

```

import socket
import sys

HOST = "<ROBOT_IP_ADDRESS>" # replace with your robot's actual IP
PORT = 5050

VALID_MEDS = ["med1", "med2", "med3"]
VALID_BEDS = ["bed1", "bed2", "bed3"]

def print_help():
    print("\nCommand format: move medX to bedY")
    print("  X = 1, 2, or 3")
    print("  Y = 1, 2, or 3")
    print("Example: move med1 to bed1")
    print("Type 'quit' to exit.\n")

```

```

def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.settimeout(180) # allow up to 3 minutes for a full delivery

    print("Connecting to robot at %s:%d..." % (HOST, PORT))
    client.connect((HOST, PORT))
    print("Connected.\n")
    print_help()

    while True:
        cmd = input("Command: ").strip().lower()

        if cmd == "quit":
            break

        if cmd == "help":
            print_help()
            continue

        print("Sending: %s" % cmd)
        print("Robot is busy – please wait, this may take a minute or two...")

        client.send(cmd.encode("utf-8"))

        try:
            response = client.recv(1024).decode("utf-8").strip()
            if response.startswith("OK"):
                print("SUCCESS: %s\n" % response)
            else:
                print("FAILED: %s\n" % response)
        except socket.timeout:
            print("No response received (timeout). Check robot status manually.\n")

        client.close()

if __name__ == "__main__":
    start_client()

```

Replace `<ROBOT_IP_ADDRESS>` with the robot's actual IP address on your WiFi network.

9.5 Understanding the busy lock

Notice that `client.recv(1024)` in the Mac script is a *blocking* call — the script pauses on that line until the robot sends a response, and the `input("Command: ")` prompt for the next command doesn't appear until that response arrives. This is the "busy lock": it's structurally impossible to send a second command while waiting for the first one's result, because the program hasn't reached the next `input()` line yet.

The `settimeout(180)` line (180 seconds = 3 minutes) prevents the Mac from waiting forever if something goes wrong on the robot side — if no response arrives within 3 minutes, it prints a timeout message and lets you try again. Adjust this value if your deliveries reliably take longer or shorter than that.

9.6 Running the connected system

On the robot, make sure `roscore`, `jetauto_bringup`, and `jetauto_nav` (with the map

loaded and localized, per Chapters 2-4) are all running. Then start the server:

```
python3 ~/chat_server.py
```

You should see `Waiting for connection on port 5050...`

On your Mac, in a new terminal:

```
python3 chat_client.py
```

You should see `Connected.` followed by the help text and a `Command:` prompt.

9.7 First end-to-end test

Type:

```
move med1 to bed1
```

On the Mac, you'll see `Robot is busy — please wait....` On the robot's terminal, you'll see the step-by-step printouts from `delivery.py` (Chapter 8) as the robot navigates, detects, picks, navigates, and drops. When finished, the Mac should print:

```
SUCCESS: OK: Delivered med1 to bed1
```

If you see this, the entire pipeline — Mac, socket, robot, navigation, detection, arm — is working together correctly for the first time.

9.8 End-of-chapter checklist

- ☐ `chat_server.py` updated on robot to call `execute_delivery()`
- ☐ `chat_client.py` updated on Mac with correct robot IP address and busy-lock behavior
- ☐ Server started on robot, shows "Waiting for connection..."
- ☐ Client connected from Mac successfully
- ☐ `move med1 to bed1` runs the full pipeline and Mac displays `SUCCESS: OK: Delivered med1 to bed1`

If all boxes are checked, proceed to Chapter 10.

Chapter 10: Full Testing — All Combinations

10.1 What this chapter achieves

This chapter is about reliability, not new features. You will systematically test every combination of medicine and bed (9 total, since you have 3 medicines and 3 beds), record the results, and fix any issues found — so that by the end, your system works consistently, not just "sometimes."

10.2 The 9 combinations

#	Command	Medicine	Bed
1	move med1 to bed1	med1	bed1
2	move med1 to bed2	med1	bed2
3	move med1 to bed3	med1	bed3
4	move med2 to bed1	med2	bed1
5	move med2 to bed2	med2	bed2
6	move med2 to bed3	med2	bed3
7	move med3 to bed1	med3	bed1
8	move med3 to bed2	med3	bed2
9	move med3 to bed3	med3	bed3

10.3 Setting up a results log

Before testing, create a simple table (on paper, or in a spreadsheet) with these columns: Combination number, Command, Navigation to shelf (pass/fail), Detection (pass/fail), Pick (pass/fail), Navigation to bed (pass/fail), Drop (pass/fail), Final Mac message, Notes.

You'll fill in one row per test run. For each combination, run it at least once; if time allows, run each combination 2-3 times to catch intermittent issues.

10.4 Running the tests

For each combination:

1. Make sure the correct medicine item is placed at its marked shelf spot (Chapter 1, step 1.5) — only the medicine being tested needs to be present, though having all 3 present at once is also fine since your model handles multi-detection (Chapter 6).
2. Make sure the robot is at its starting position (Chapter 1, step 1.4) and localized (Chapter 4, step 4.3).
3. On the Mac, type the command for this combination.
4. Watch each step on the robot's terminal and physically observe the robot.
5. Fill in your results log based on what you observed at each step, and the final message shown on the Mac.

10.5 Interpreting and fixing failures

Navigation to shelf or bed fails (robot doesn't arrive, or arrives in the wrong spot):

Revisit Chapter 5. Confirm the robot was correctly localized before starting (Chapter 4, step 4.3 — if the robot was moved by hand at any point without re-localizing, its position estimate will be wrong). If localization was correct but navigation still fails for a specific location, re-record that location's coordinates (Chapter 4) with the robot very precisely positioned.

Detection fails (medicine not found within retries):

Revisit Chapter 6. Check lighting hasn't changed since your last successful test (Chapter 1, step 1.6). Check the medicine item is at its marked position and the camera has a clear view (nothing blocking it). If detection is consistently failing for one specific medicine but not others, that medicine's item may have moved slightly from the position used in your 300 training images — try repositioning it, or consider lowering the confidence threshold slightly (Chapter 6, step 6.9).

Pick fails (arm misses, or item slips):

Revisit Chapter 7. Re-check that specific medicine's recorded positions in `arm_positions.json`. Small position drift (a few millimeters) can sometimes occur if the shelf or robot was bumped — re-record that medicine's `_approach`, `_grip`, and `_lift` positions if needed.

Drop fails (item doesn't land in bed area, or is dropped early):

Revisit Chapter 7, step 7.9's notes on per-bed drop positions. If this only fails for specific beds, that's a sign you need per-bed drop positions rather than one shared position — record separate `drop_approach/release` positions for the problematic bed(s) and update `arm_control.py`'s `drop()` function to accept a bed name parameter and look up the bed-specific position.

Socket/Mac issues (no response, or wrong message):

Revisit Chapter 9. Check the robot's `chat_server.py` terminal for Python errors — these will often point directly at which underlying function (navigator/detector/arm_control) raised an exception, which then tells you which earlier chapter to revisit.

10.6 Re-testing after fixes

After fixing any issue, re-run that specific combination (not all 9) to confirm the fix worked, then continue with the remaining combinations. Once a fix is made to a shared component (e.g. a navigation location, or the parser), it's worth spot-checking one or two *other* combinations that use that same component too, to make sure the fix didn't break something else.

10.7 Target outcome

Aim for all 9 combinations passing cleanly, with consistent results across repeated runs (2-3 times each) for at least the combinations you plan to demonstrate live. If 1-2 combinations remain unreliable despite troubleshooting, document this honestly in your report (Chapter 11) — explaining what was tried and why it remains an issue is valuable and expected in a capstone, and is far better than hiding it.

10.8 End-of-chapter checklist

- ☐ Results log created with all 9 combinations listed
- ☐ All 9 combinations tested at least once, results recorded
- ☐ Any failures investigated using the relevant earlier chapter, and fixes applied

- ☐ Fixed combinations re-tested to confirm
- ☐ At least the combinations planned for the live demo pass reliably across 2-3 repeated runs

If all boxes are checked, proceed to Chapter 11.

Chapter 11: Demo and Final Submission

11.1 What this chapter achieves

This final chapter covers preparing your recorded demo video, cleaning up your code for submission, and structuring your written report. By the end, you have everything needed to submit the capstone.

11.2 Planning the demo video

Decide in advance which 3 (minimum) of the 9 combinations from Chapter 10 you will demonstrate — choose ones that passed reliably and repeatedly. A good spread is to choose combinations that use all 3 medicines and all 3 beds across your chosen set, e.g. `move med1 to bed1`, `move med2 to bed2`, `move med3 to bed3` (or any 3 combinations covering all medicines and all beds at least once).

11.3 Recording the video

Set up a camera (phone camera is fine) on a tripod or stable surface, positioned to capture the entire 3x3 room area — wide enough to see the shelf, all 3 beds, and the robot's full path between them. Also have a second view (or a second short clip) of the Mac screen showing the command being typed and the response received.

For each of your chosen combinations, record continuously from before you type the command until the Mac shows the final **SUCCESS:** message. Don't cut/edit out any part of the robot's actual movement — a continuous, unedited run is what demonstrates the system genuinely works autonomously.

Between each combination, you can pause recording while you reset the medicine item to its shelf position (since the robot delivered it to a bed) and return the robot to its starting position — but make sure each individual combination's run, from command to confirmation, is one continuous unedited clip.

At the start of the video, include a short clip (10-15 seconds) showing the overall room layout — shelf, beds, robot starting position — so viewers understand the physical setup before watching the deliveries.

11.4 Code cleanup

Before final submission, go through each script created in this book (`navigator.py`, `detector.py`, `arm_control.py`, `parser.py`, `delivery.py`, `chat_server.py`, `chat_client.py`) and:

- Remove any leftover test/debug code that isn't part of the final flow (e.g. the standalone test blocks used in Chapter 6, step 6.4, if you no longer need them)
- Make sure file paths (`locations.json`, `arm_positions.json`, API keys) are either correct for the robot's actual filesystem, or — better — moved to a single small configuration file/section at the top of each script so a reviewer can see all the "settings that need to match your setup" in one place
- Remove your actual Roboflow API key from any code you submit publicly — replace it with a placeholder and a clear instruction comment (e.g. `API_KEY = "YOUR_API_KEY_HERE" # insert your own Roboflow API key`)

Confirm the entire system can be started with just two commands:

- On the robot: one command (or a short documented sequence) that starts `roscore`, `jetauto_bringup`, `jetauto_nav` (with map), and `chat_server.py`
- On the Mac: `python3 chat_client.py`

If currently this requires many separate terminal windows (which is normal given everything in this book), consider writing a single shell script on the robot that launches each piece in the background in sequence — this is a nice polish item but not strictly required if your report clearly documents the startup sequence instead.

11.5 Report structure

Your written report should cover the following sections, each summarizing the corresponding chapters of this book:

System overview — one paragraph describing the full flow: Mac command → socket → robot → navigation → detection → arm → confirmation. Include a simple diagram showing these components and how they connect (this can be a simple boxes-and-arrows drawing).

Physical setup — summarize Chapter 1: room dimensions, fixed positions of shelf/beds/robot start, and why these positions were chosen to remain fixed throughout the project.

Mapping and navigation — summarize Chapters 3-5: what SLAM is and how the map was built, how goal locations were recorded, and how `go_to()` works using `move_base`.

Machine learning component — this is your core AI contribution. Describe: the Roboflow platform and workflow used, the dataset (300 custom images across 3 medicine classes), the model type (object detection), the achieved accuracy (90%+), and how the trained model was deployed for inference on the robot (local inference, as set up in Chapter 6). Include example detection results (screenshots of the prediction output with confidence scores) if possible.

Robotic arm control — summarize Chapter 7: the fixed-position approach, why it was chosen, and how pick/drop functions work.

Integration — summarize Chapters 8-9: the command parser, the full delivery sequence, and the socket-based connection between Mac and robot.

Testing and results — present your Chapter 10 results table (all 9 combinations, pass/fail, notes). Be honest about any combinations that remained unreliable, and explain what was tried.

Challenges and learnings — describe specific problems you ran into during this project and how you solved them (e.g. localization drift, arm position tuning, lighting affecting detection). This section is highly valued in capstone evaluation — it demonstrates genuine engineering problem-solving, not just following steps.

Future improvements — briefly mention what could be added with more time (e.g. vision-guided picking instead of fixed positions, support for more medicines by retraining the model with more classes, voice command input instead of typed text, handling multiple medicines in one delivery run).

11.6 Final submission checklist

- ☐ Demo video recorded: room overview + at least 3 combinations, each as one continuous unedited clip from command to confirmation
 - ☐ All code files cleaned up: no leftover debug code, configuration values clearly marked at the top of each file, API keys replaced with placeholders
 - ☐ Startup sequence documented clearly (either as a single script or as clearly listed steps)
 - ☐ Report written covering all 8 sections listed in 11.5
 - ☐ Chapter 10 results table included in the report, with honest notes on any remaining issues
 - ☐ Backup copies of all code files, `locations.json`, `arm_positions.json`, and the map files (`room_map.pgm/.yaml`) saved somewhere safe (e.g. your Mac and/or a USB drive), separate from the robot itself
-

End of Book

If you have followed every chapter in order and checked every box, you have a complete, working, end-to-end autonomous delivery robot system, built from an empty room to a recorded demo, with full documentation of every step — including every configuration file, every script, and every physical measurement needed to reproduce it.